

Classic Meets Modern: a Pragmatic Learning-Based Congestion Control for the Internet

Soheil Abbasloo, Chen-Yu Yen, H. Jonathan Chao
SIGCOMM '20

Motivation

State-of-the-art

- Current CCAs are either jack of all trades (too generic), or laser-focused (too precise)
- We still lack an **adaptive** CCA
 - Achieves consistent performance in different environments
 - Does not need manual tuning

Motivation

Existing learning-based CCAs

- People have built learning based CCAs before:
 - Indigo
 - Aurora
 - Vivace
 - Remy

Motivation

Existing learning-based CCAs

- Problems with learning based congestion control:
 - Don't perform well with unseen network scenarios
 - Converge on wrong equilibrium point or may converge slowly
 - High computational overhead

Motivation

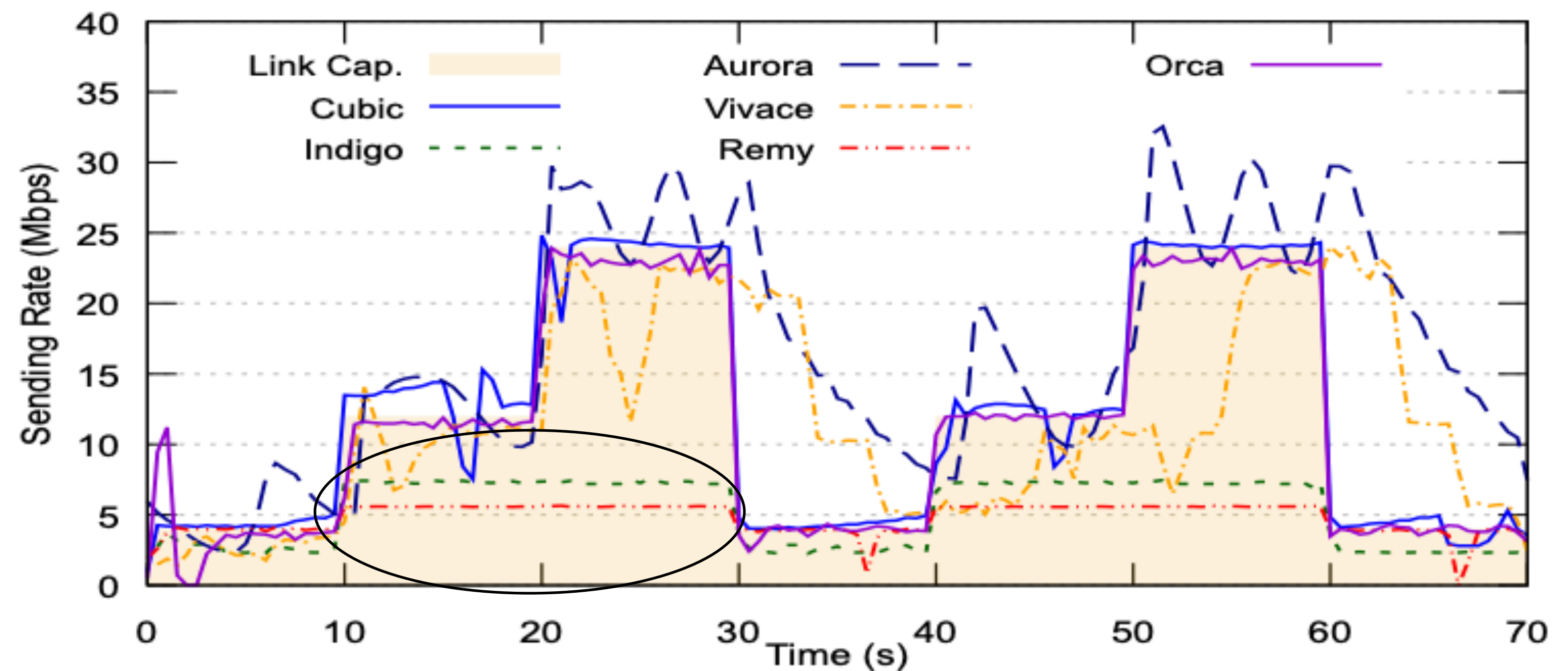
Motivational Experiment

- Emulate a network with a minimum RTT of 20ms
- Bottleneck link bandwidth changes every 10 seconds
- Enough buffer space to absorb the traffic (= 1.5MB)

Motivation

Motivational Experiment

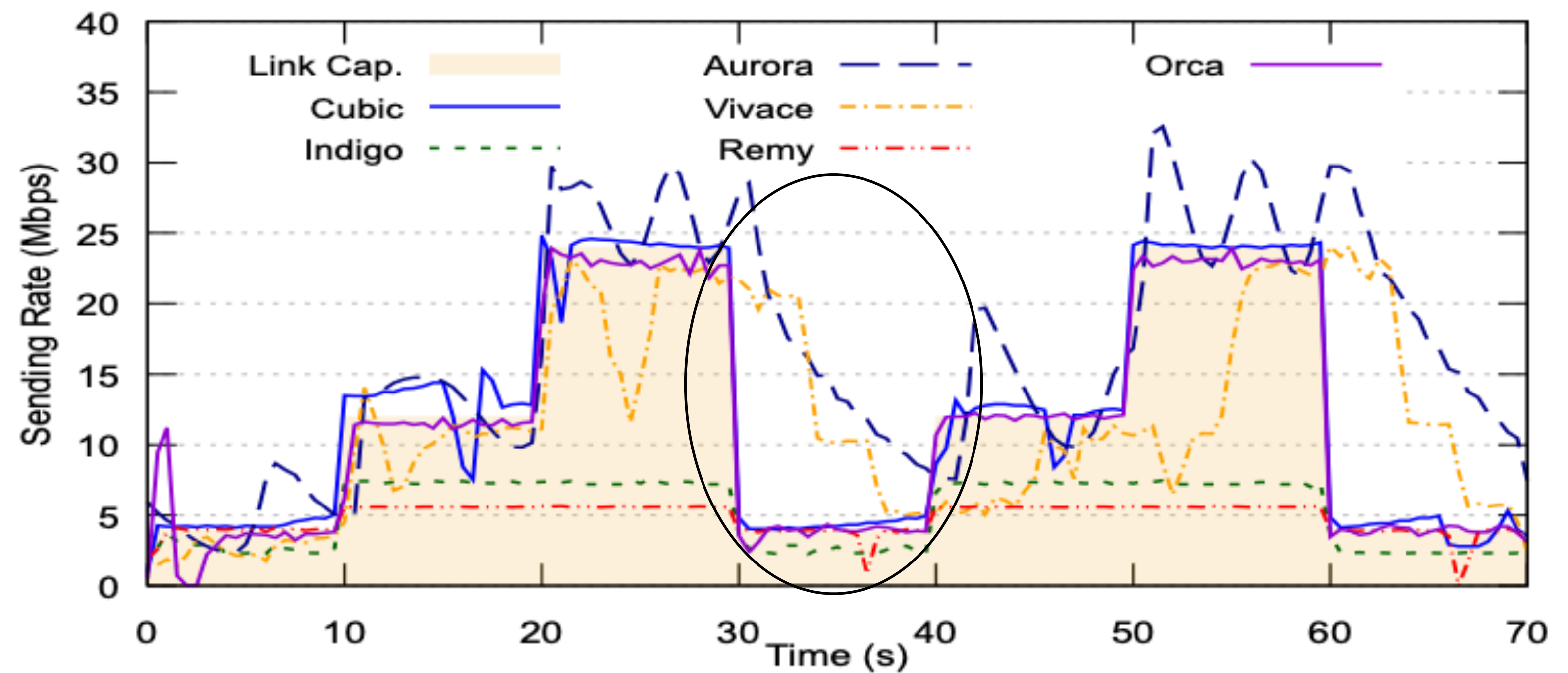
Indigo and Remy under-utilize the network



Motivation

Motivational Experiment

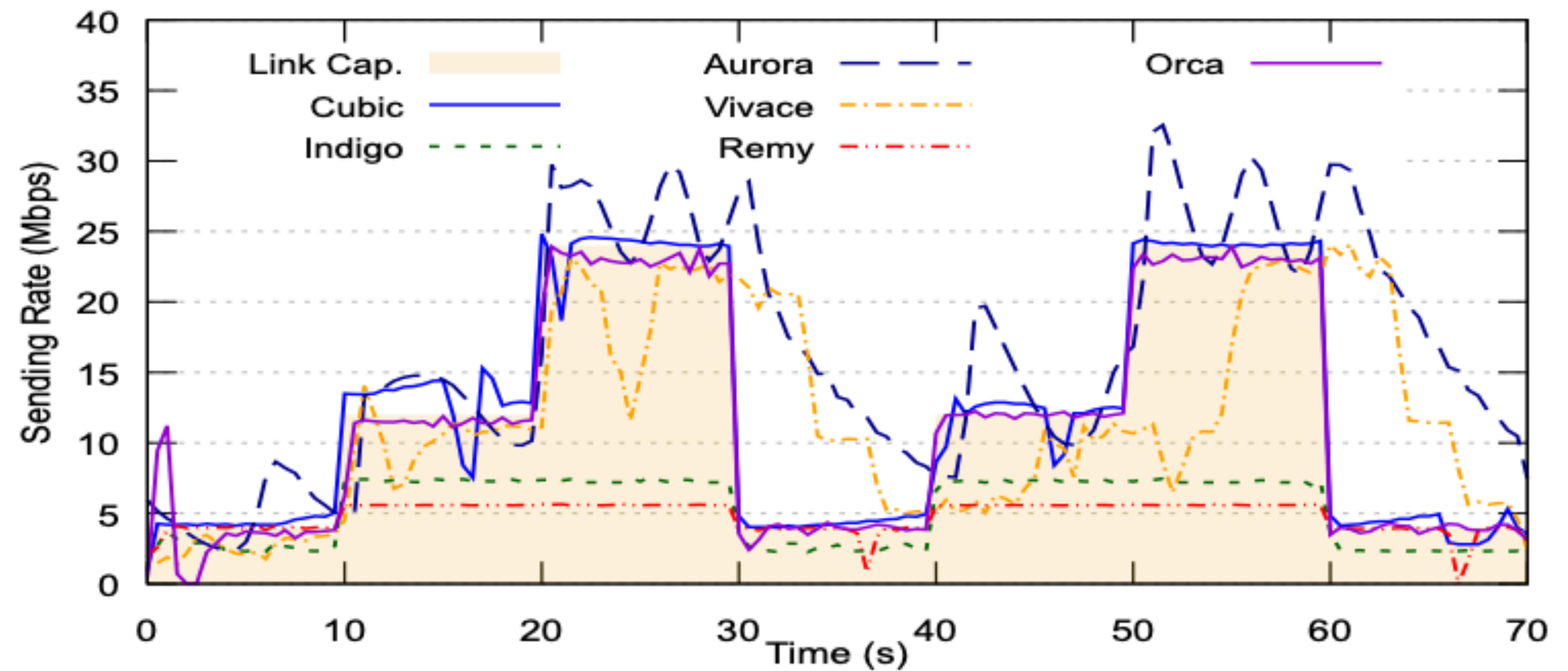
Aurora and Vivace over-utilize the network



Motivation

Motivational Experiment

Aurora does not converge



Design Overview

Two level control

- Mix of classic and modern design
- Fine grained: Use the underlying congestion control scheme
- Monitoring using DRL at a coarser granularity

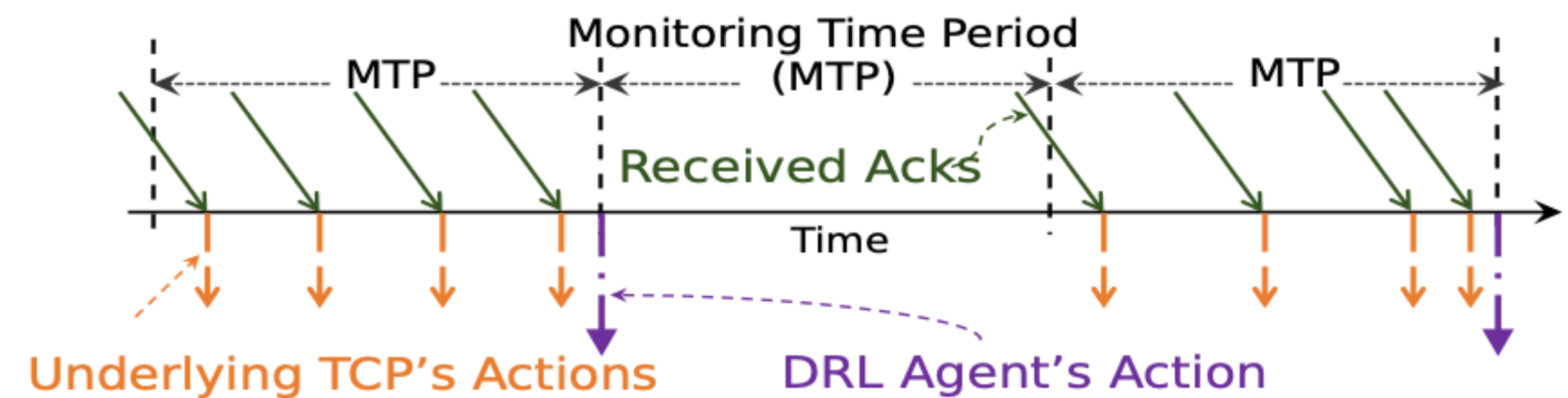


Figure 2: Simplified time-diagram of Orca's two-Level control logic

Orca's Design

Monitoring Block

- Core Function: The "Eyes" of the System
- Architecture: Kernel shim layer
- Data Processing: Normalization & Recurrent Structure

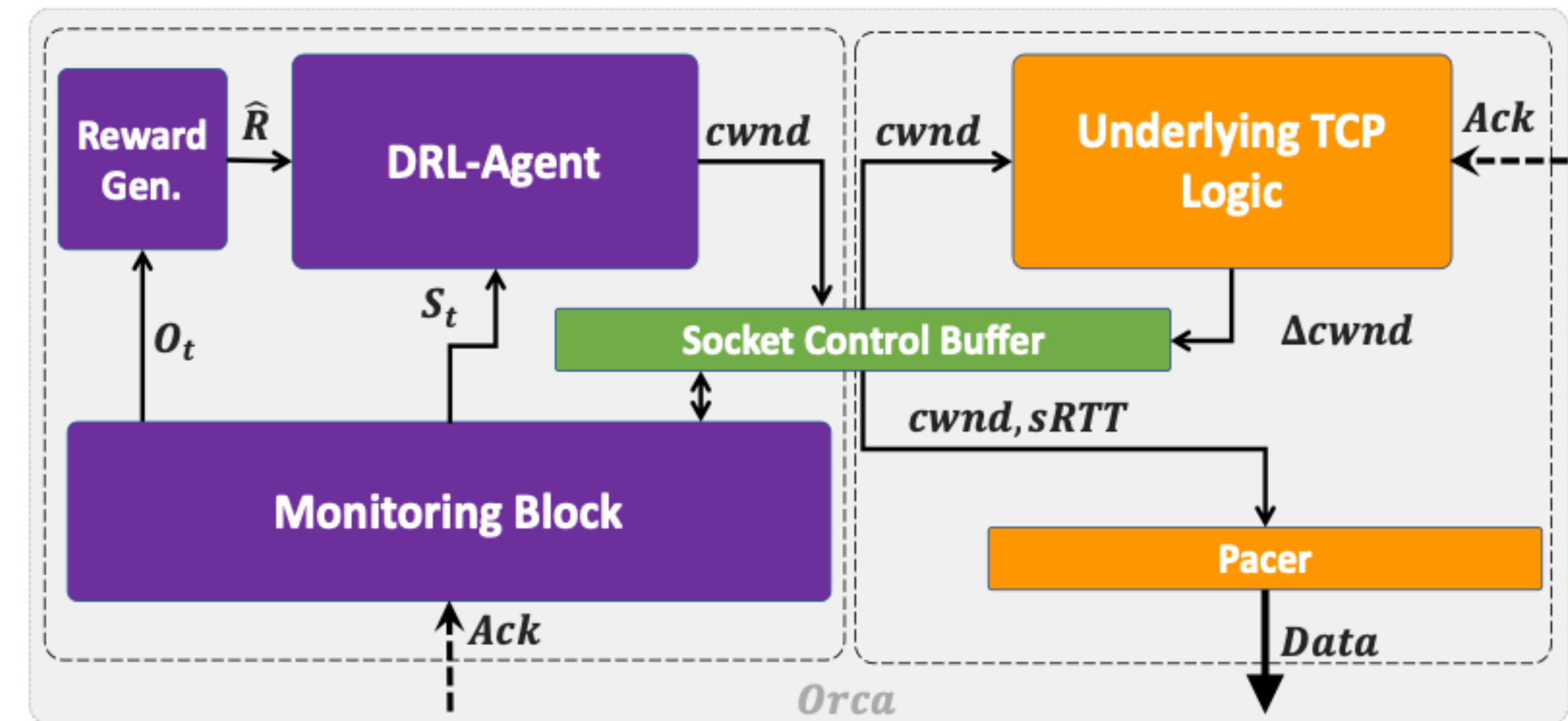


Figure 3: High-level block diagram of Orca

Orca's Design

Reward

- Maximize Power
 - Power is Throughput / Delay
- Hard to measure power though
 - Use $\beta = 1.25$

$$\hat{R} = R/R_{max} = (\frac{throughput - \zeta \times loss}{delay'}) / (\frac{thr_{max}}{d_{min}})$$

$$delay' = \begin{cases} d_{min} & (d_{min} \leq delay \leq \beta \times d_{min}) \\ delay & \text{o.w.} \end{cases}$$

Orca's Design

DRL-Agent

- Core component
 - Takes actions based on rewards—congestion window and pacing rate
- Congestion Window
 - Use underlying TCP's $cwnd_u$
 - Alpha is DRL-Agent determined
- Pacing rate
 - sRTT is measured by the monitoring block

$$cwnd = 2^{\alpha} \times cwnd_u \quad (-2 < \alpha < 2)$$

$$prate = \frac{cwnd}{sRTT}$$

Orca's Design

Efficient Distributed Learning

- Challenges in training DRL for congestion control
 - Catastrophic Forgetting
 - Extended Training Time
- Scaling Out the Learning
 - Actors (The Workers)
 - Replay Memory (The Experience Bank)
 - Learner (The Brain)

Implementation Details

- Orca is a server-side only modification
- Hybrid Architecture:
 - User-Space: The DRL-Agent is implemented using TensorFlow
 - Kernel-Space: The Monitoring Block and control logic are implemented as a shim layer in the Linux kernel
- Underlying TCP: Uses TCP Cubic

Training Architecture

Distributed Actors-Centralized Learner

- Actors
 - Over 256 actors run in parallel, each interacting with a different emulated network environment using Mahimahi
- Learner: A centralized unit, updates the neural network models using data collected by the actors
- Global Replay Memory: A shared database that stores "experience tuples" (state, action, reward) from all actors for the learner to sample

Training Results

- Orca vs. Clean-Slate: "Clean-Slate" version that has no underlying TCP control.
- Speed of Learning: Orca's hybrid approach allows it to learn 2x faster than the clean-slate version in the same amount of time.

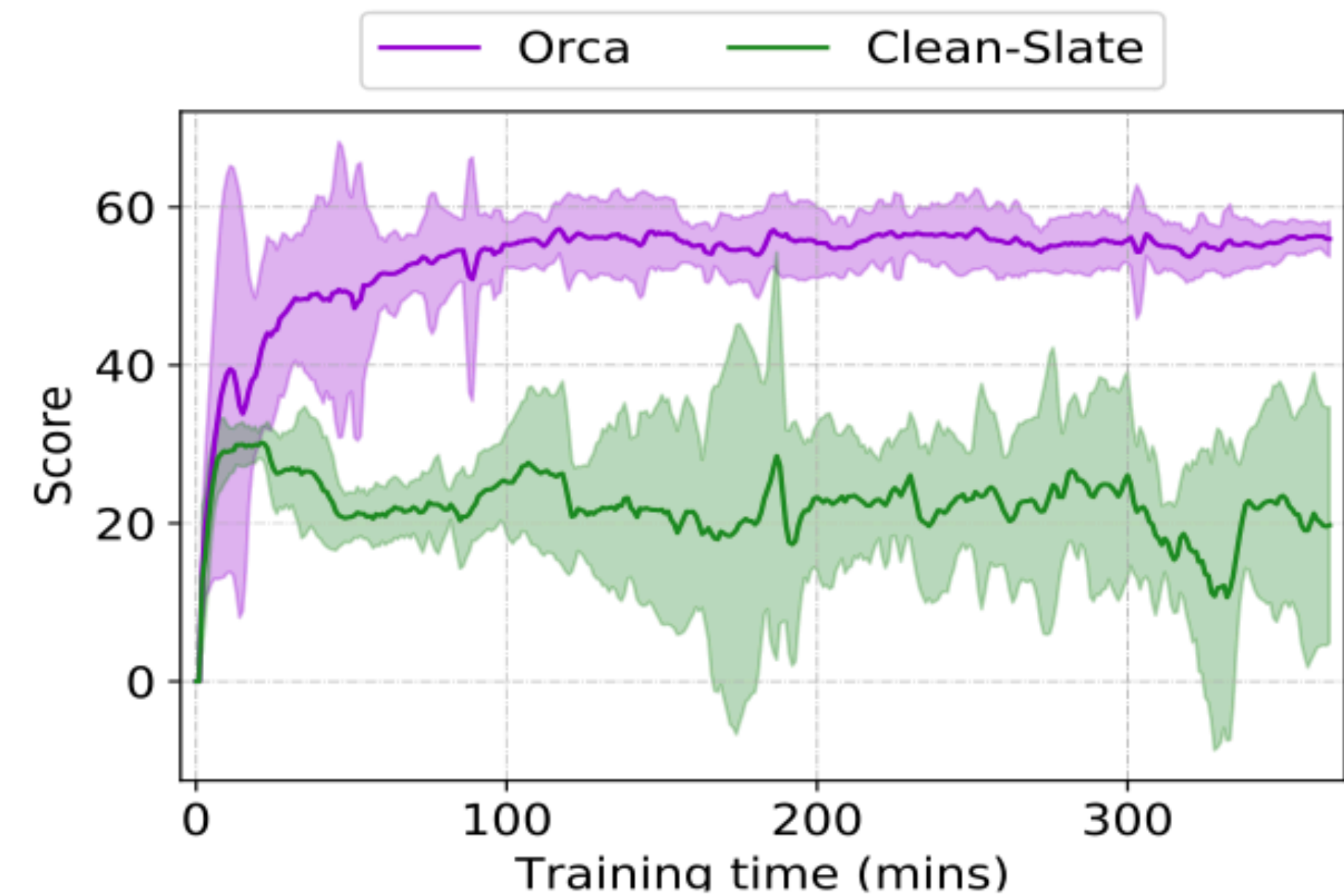
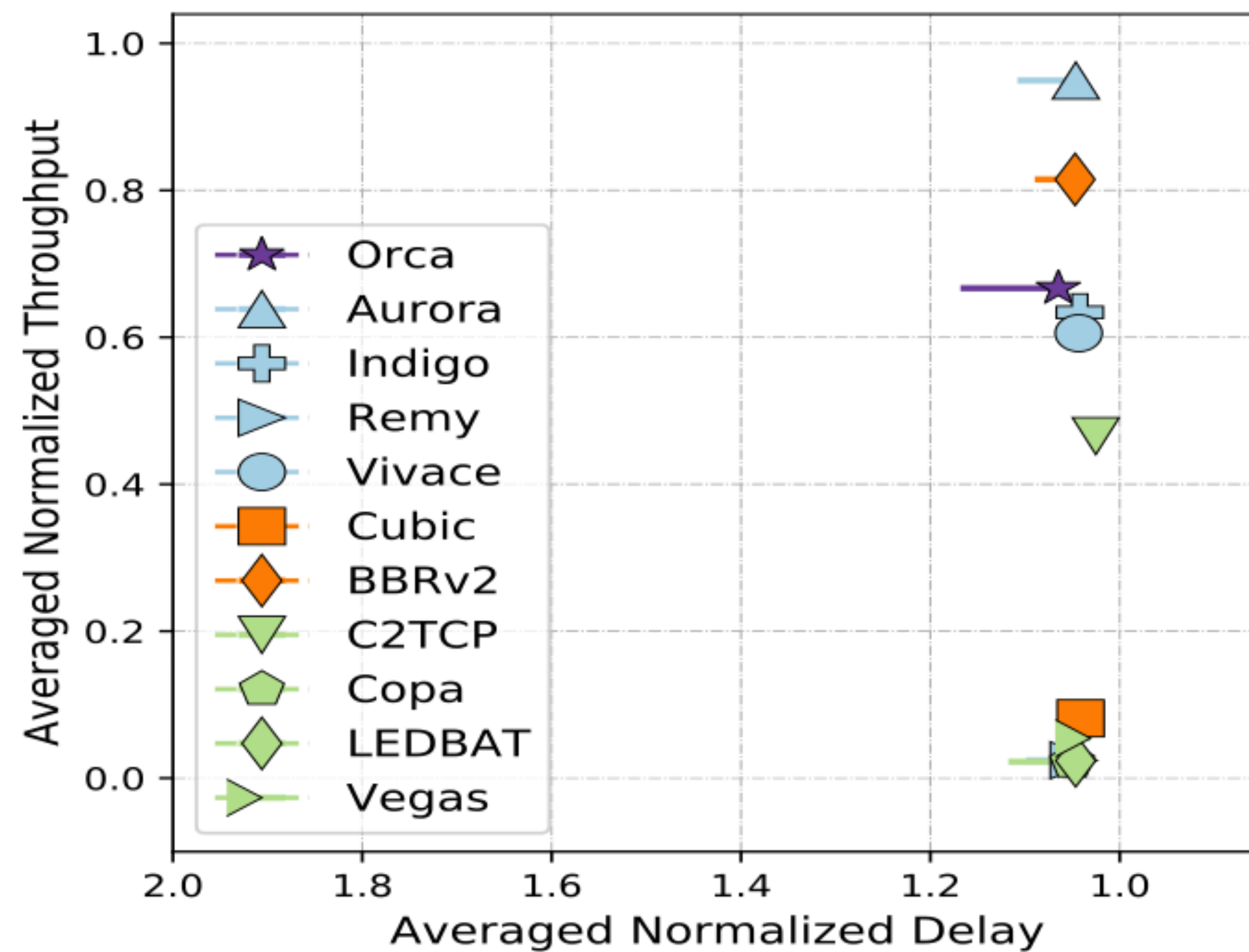


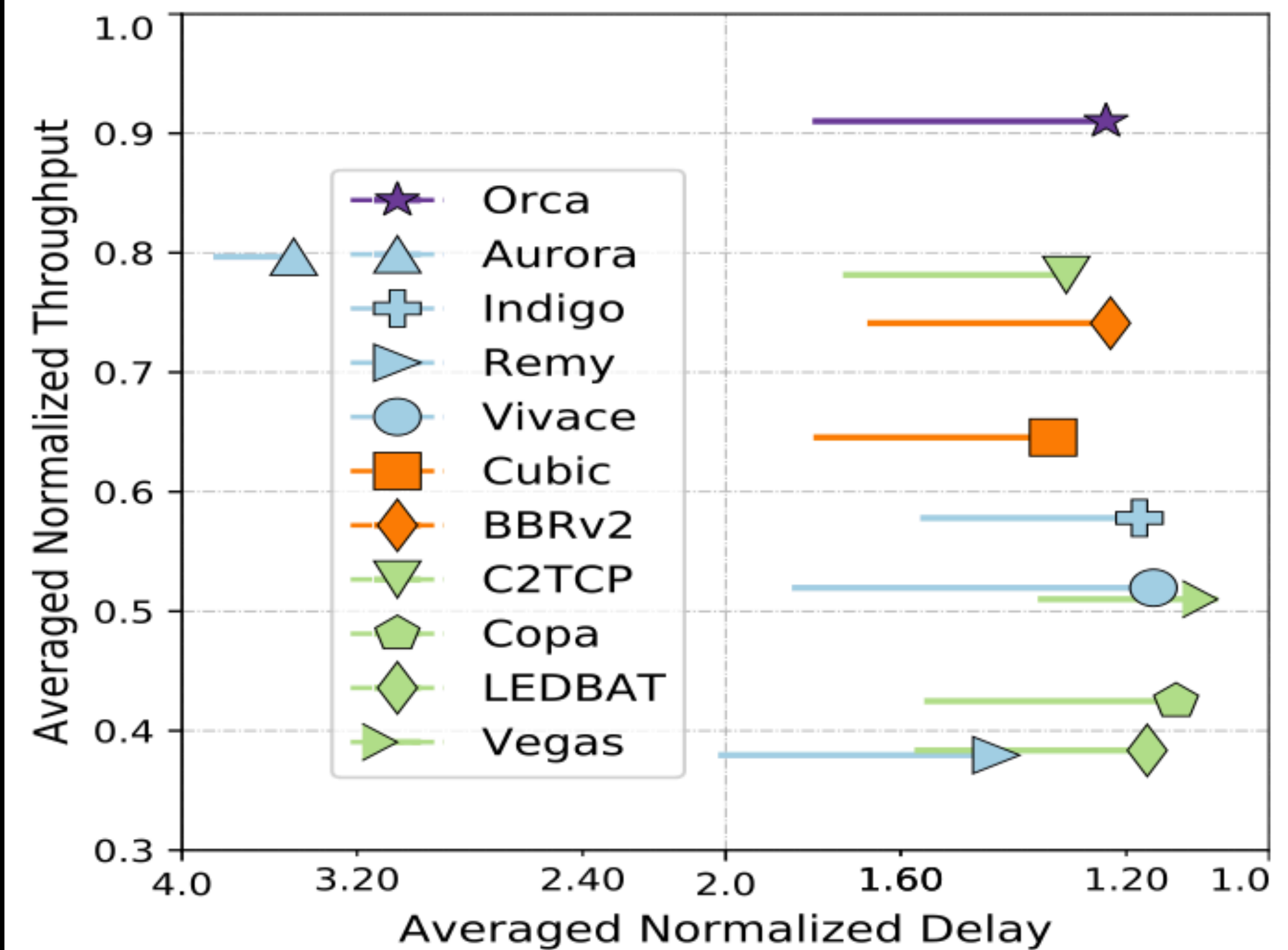
Figure 4: Training curves of Orca and Clean-Slate (averaged over all sessions)

Evaluation

Inter-continental and Intra-continental scenarios



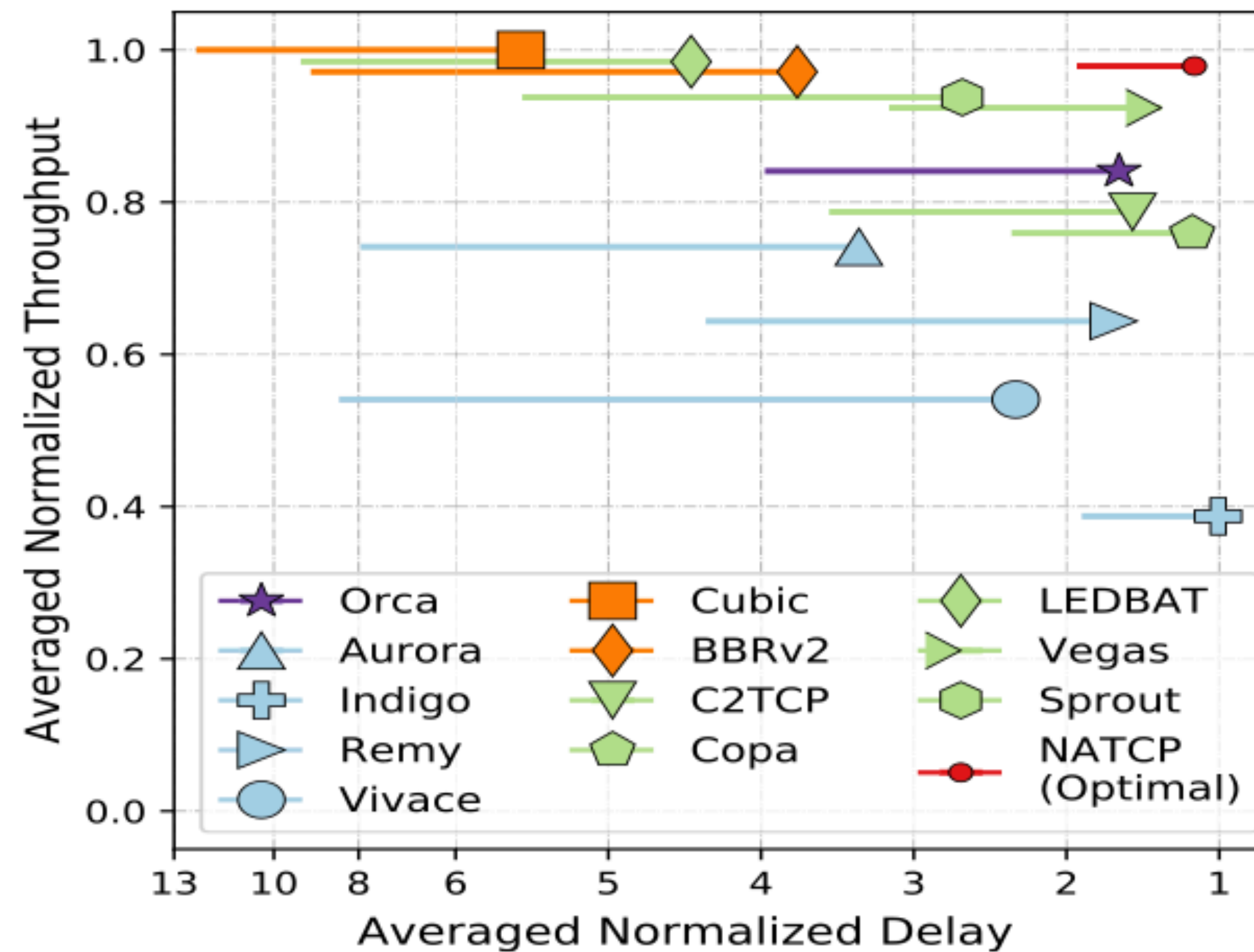
(a) Inter-Continental experiments



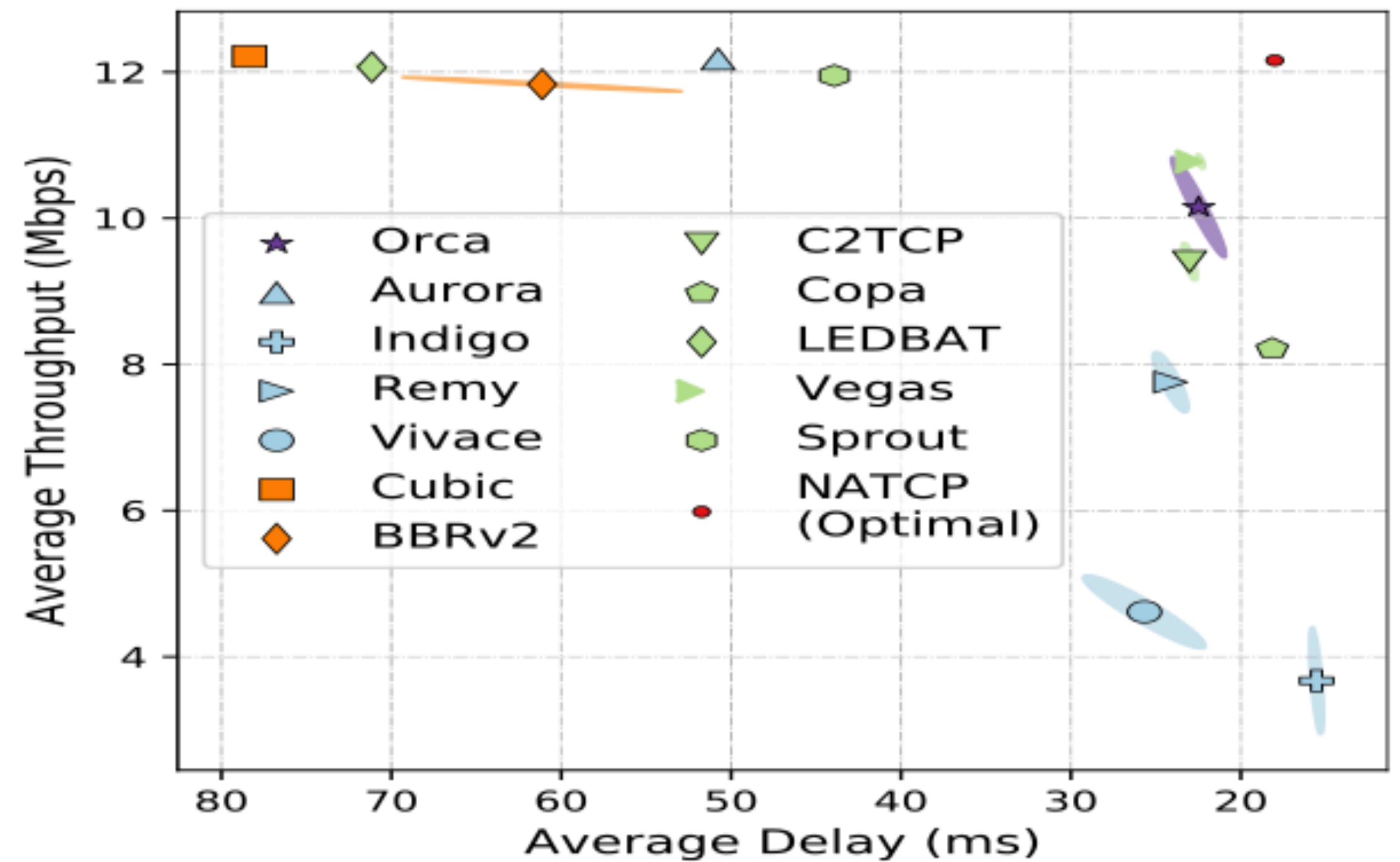
(b) Intra-Continental experiments

Evaluation

Highly variable link experiments



(c) Highly variable link experiments



(c) Cellular LTE (Times-Square Trace)

Evaluation

CPU Overhead

- Orca has the lowest CPU overhead compared to other learning based CCAs

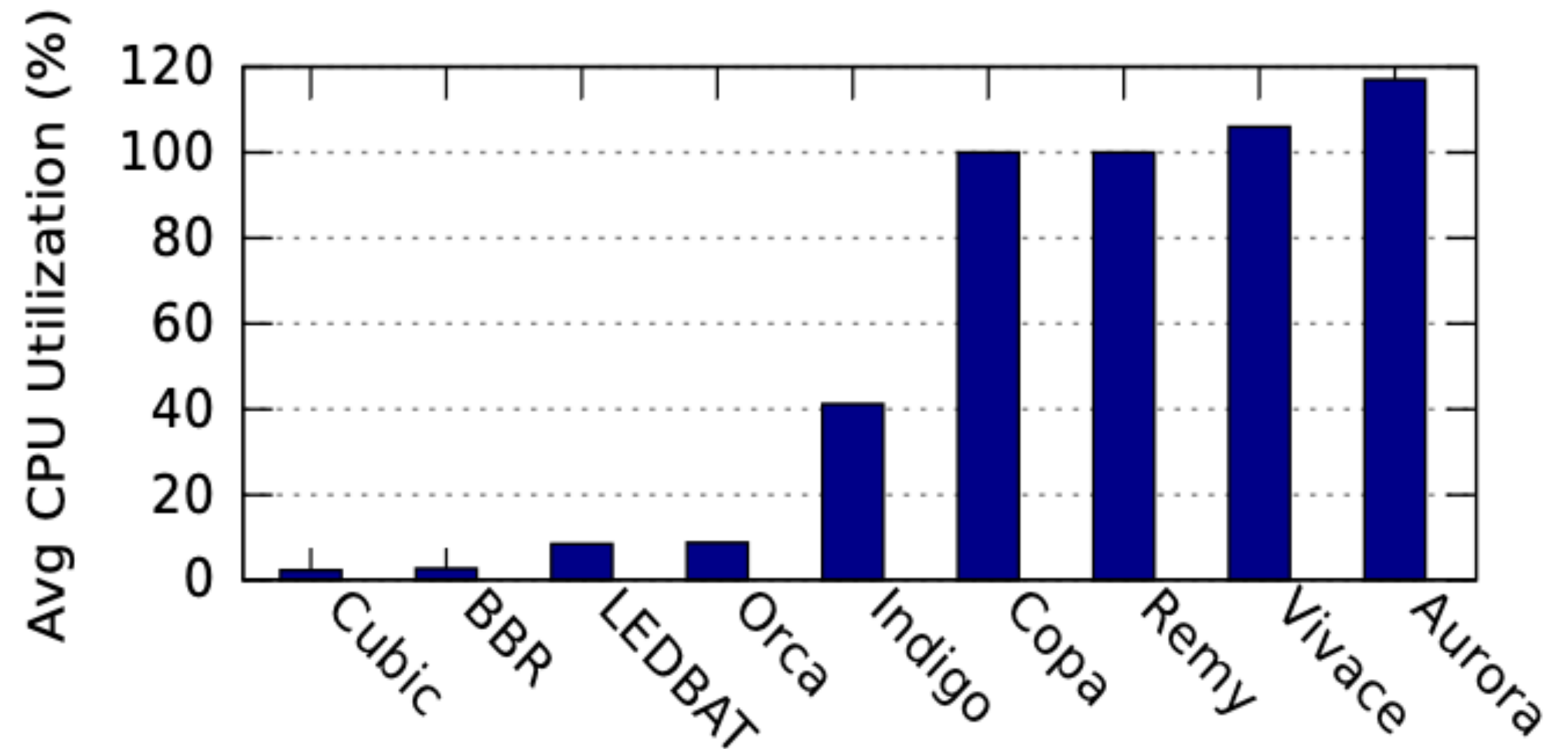


Figure 10: Overhead of DRL agent compared with other schemes

Evaluation

CPU Overhead and MTP

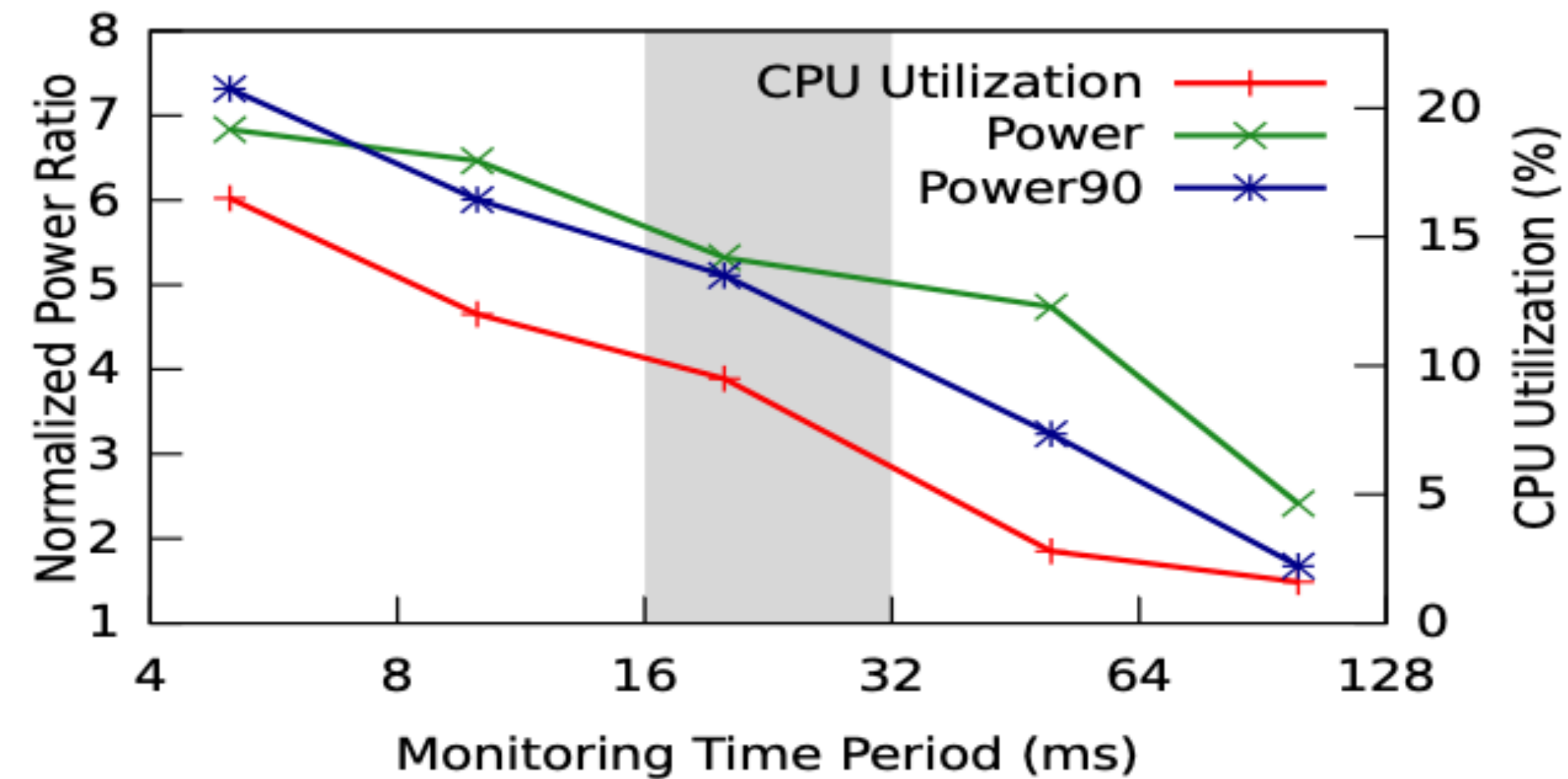


Figure 11: Performance & overhead trade-off across diff. MTPs

Evaluation

Deployment Friendliness

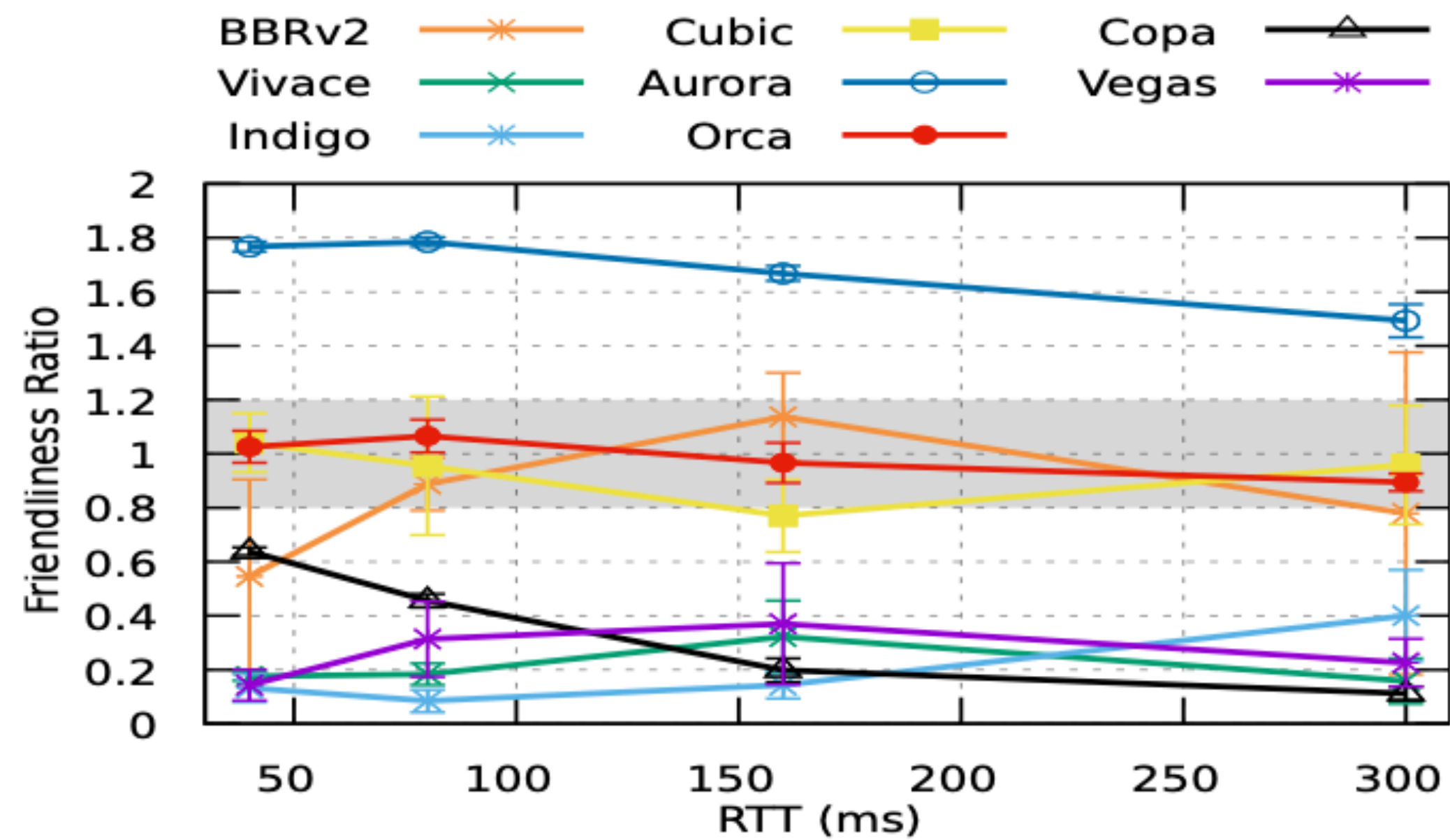


Figure 14: Friendliness Ratio of schemes across different RTTs

- Orca achieves good friendliness

Reflection

- Unique design point — they combined a classic CCA with a learned CCA
 - Reduced overhead by coarse granularity checking
 - Use the underlying TCP's cwnd as a base to reduce search space
- Broad evaluation — they use inter/intra and highly variable links
- The throughput and delay experiments can use more flows to be more realistic